

Become a Guru in Java

Author: Luv, Priya

Description:

Become a Guru in Java is an advanced guide for developers looking to master Java programming. This book dives deep into the core concepts of Java. This book will help you achieve a high level of proficiency in Java, making you a true Java guru.

TABLE OF CONTENTS

1. Java Fundamentals Review
2. Object-Oriented Programming (OOP) in Depth
3. Classes and Objects Mastery
4. Inheritance and Polymorphism
5. Abstraction and Interfaces
6. Exception Handling
7. Collections Framework
8. Generics in Java
9. File Handling and I/O
10. Multithreading and Concurrency
11. Java Streams and Lambda Expressions
12. Design Patterns in Java
13. Java Memory Management
14. Building Real-World Java Applications
15. Conclusion

1. JAVA FUNDAMENTALS REVIEW

Java is a high-level, object-oriented programming language designed to be platform-independent through the Java Virtual Machine (JVM).

Key features:

- Object-oriented
- Platform independent
- Secure and robust
- Multithreaded

Example:

```
public class Main {  
    public static void main(String[] args) {  
        System.out.println("Welcome to Java Guru Level");  
    }  
}
```

2. OBJECT-ORIENTED PROGRAMMING (OOP) IN DEPTH

Java is built around OOP principles:

- Encapsulation
- Inheritance

- Polymorphism
- Abstraction

OOP allows developers to structure code in a modular and reusable way.

3. CLASSES AND OBJECTS MASTERY

A class is a blueprint, and an object is an instance of a class.

Example:

```
class Car {
String model;
```

```
void start() {
    System.out.println(" Car started");
}
```

```
}
Car myCar = new Car();
myCar.start();
```

4. INHERITANCE AND POLYMORPHISM

Inheritance allows one class to inherit properties of another.

Example:

```
class Animal {
void sound() {
System.out.println("Animal sound");
}
}
class Dog extends Animal {
void sound() {
System.out.println("Bark");
}
}
```

Polymorphism allows methods to behave differently based on the object.

5. ABSTRACTION AND INTERFACES

Abstraction hides implementation details.

```
abstract class Vehicle {
abstract void run();
}
```

Interface example:

```
interface Animal {
void sound();
}
```

Interfaces define behavior without implementation.

6. EXCEPTION HANDLING

Used to handle runtime errors safely.

Example:

```
try {  
    int result = 10 / 0;  
} catch (ArithmeticException e) {  
    System.out.println("Error: Division by zero");  
}
```

7. COLLECTIONS FRAMEWORK

Java provides built-in data structures:

- List
- Set
- Map

Example:

```
List<String> names = new ArrayList<>();  
names.add("John");  
Collections make data management efficient.
```

8. GENERICS IN JAVA

Generics allow type-safe code.

Example:

```
class Box<T> {  
    T value;  
}  
Box<Integer> box = new Box<>();
```

9. FILE HANDLING AND I/O

Used to read and write files.

Example:

```
File file = new File("data.txt");  
FileWriter writer = new FileWriter(file);  
writer.write("Hello Java");  
writer.close();
```

10. MULTITHREADING AND CONCURRENCY

Java supports parallel execution.

Example:

```
class MyThread extends Thread {  
    public void run() {  
        System.out.println("Thread running");  
    }  
}
```

Threads improve performance.

11. JAVA STREAMS AND LAMBDA EXPRESSIONS

Streams simplify data processing.

Example:

```
List<Integer> numbers = Arrays.asList(1, 2, 3, 4);  
numbers.stream().filter(n -> n % 2 == 0).forEach(System.out::println);
```

12. DESIGN PATTERNS IN JAVA

Common patterns:

- Singleton
- Factory
- Observer
- Strategy

They help solve common software design problems.

13. JAVA MEMORY MANAGEMENT

Java uses garbage collection to manage memory automatically.

Memory areas:

- Heap
- Stack

Understanding memory improves performance optimization.

14. BUILDING REAL-WORLD JAVA APPLICATIONS

Real-world Java is used in:

- Web applications (Spring Boot)
- Android apps
- Enterprise systems

Example structure:

- Controller layer
- Service layer
- Repository layer

15. CONCLUSION

Mastering Java requires understanding both fundamentals and advanced concepts.

By learning OOP, collections, multithreading, and design patterns, you reach a professional level of development.

Becoming a Java guru is not just about writing code, but about writing clean, scalable, and efficient software.